SCS 2312- Computational Models and Programming Language Concepts

Take-home Assignment

Name : R.A.K.D.Dasundi
Index Number : 23000262

Introduction

This program reads a source file containing a simple arithmetic program.
It parses the code using the defined context-free grammar (CFG), and it performs lexical, syntax, and semantic analysis, executing arithmetic expressions.
The program supports variable declarations(int), addition expressions, and print statements.

Context Free Grammar (CFG)

Grammar Definition

<Program> → <StatementList>

<StatementList> → <Statement> <StatementList>
| ε

<Statement> → <Declaration>
| <PrintStatement>

<Declaration> → int <identifier> = <Expression> ;

<PrintStatement> → print (<identifier>) ;

<Expression> → <Term> <Expression'>

<Expression'> → + <Term> <Expression'>
| ε

<Term> → <identifier>
| <number>

Start Symbol

<Program>

Terminals

int print identifier number + = ; ( )

Non- Terminals

<Program> <StatementList> <Statement> <Declaration> <PrintStatement>
<Expression> <Expression'> <Term>

Implementation Overview

The program is implemented in C and consists of four main components:

1. Tokenizer (Lexical Analyzer): Converts input characters into tokens.
2. Parser (Syntax Analyzer): Analyzes the token sequence according to the CFG.
3. Symbol Table & Semantic Checking: Stores variable names and their integer values.
4. Expression Evaluation: Computes arithmetic expressions during parsing.

Tokenizer (Lexical Analyzer)

Token Types :

TOKEN_INT
TOKEN_PRINT
TOKEN_IDENTIFIER
TOKEN_NUMBER
TOKEN_PLUS
TOKEN_ASSIGN
TOKEN_SEMICOLON
TOKEN_LPAREN
TOKEN_RPAREN
TOKEN_EOF
TOKEN_INVALID

Identifiers: Start with a letter, followed by letters/digits

Numbers: Sequence of digits

Keywords: int and print

Function used: getNextToken() function reads characters and assigns token types.

Parser (Syntax Analyzer)

Implemented using a top-down approach.
Each CFG non-terminal corresponds to a C function:

Grammar Rule	C Function
<Program>	Program()
<StatementList>	StatementList()
<Statement>	Statement()
<Declaration>	Declaration()
<PrintStatement>	PrintStatement()
<Expression>	Expression()
<Term>	Term()

match() Function

The match() function checks if the current token is the expected one.

- If correct : it moves to the next token.
- If incorrect : it prints "Syntax Error" and stops the program.

Symbol Table & Semantic Checking

The symbol table is used to store variable names and their values.

Ex1: If **int x = 5;** is parsed, the function **storeVariable()** saves, variable name as x and value as 5.

Ex2: If variable is used before declaration (**print(x);**) , it will print :
Semantic Error: Variable 'x' not defined

Ex3: It can also redeclare the variables without errors.

Expression Evaluation

Expressions are evaluated while parsing.

int z = x + y;

In **Expression() :**

- First term is read.
- If + is found, the next term is read.
- The values are added immediately.
- The final result is returned.

In Printing results we use **PrintStatement()** :

The value is taken from the symbol table and printed.

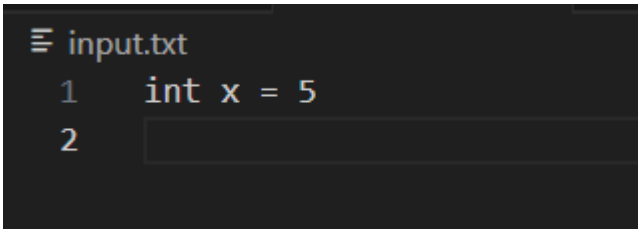
Error Handling

The program handles two types of errors.

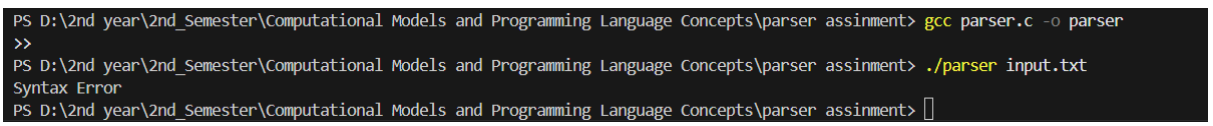
Syntax Errors

These happen when the grammar rules are broken.

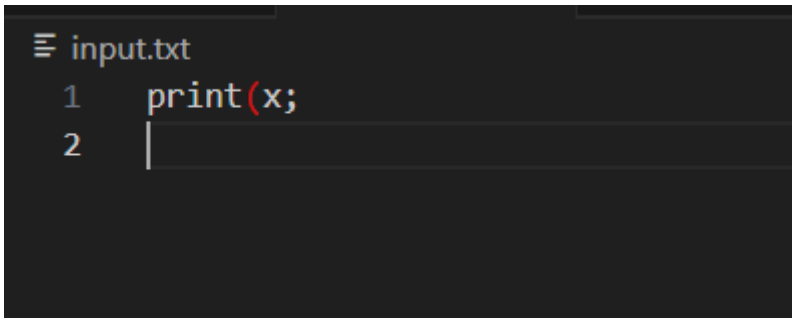
Missing Semicolon :



Output :



Missing Parenthesis :



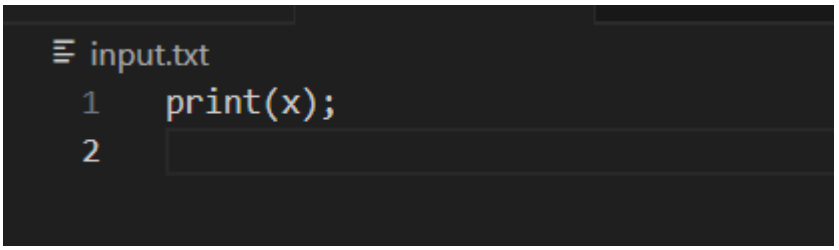
Output:

```
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> gcc parser.c -o parser
>>
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> ./parser input.txt
Syntax Error
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment>
```

Semantic Errors

These happen when the program meaning is wrong.

Undeclared Variable :



Output :

```
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> gcc parser.c -o parser
>>
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> ./parser input.txt
Semantic Error: Variable 'x' not defined
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment>
```

For the correct code :

input.txt

```
int y = 5;
int x = 20;
int z = x + y;
print(z);
```

Output :

```
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> gcc parser.c -o parser
>>
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment> ./parser input.txt
25
PS D:\2nd year\2nd_Semester\Computational Models and Programming Language Concepts\parser assinment>
```

Conclusion

In this assignment, a simple programming language parser was implemented using C language.

The program consists of:

- A lexical analyzer to produce tokens.
- A recursive descent parser based on the given CFG(Context Free Grammar).
- A symbol table to store variables and their values.
- Evaluation of expressions during parsing.
- Syntax and semantic error handling

The grammar is suitable for a top-down parser and matches the functions that were implemented.

The program successfully supports:

- Integer variable declarations
- Addition expressions
- Print statements
- Detection of syntax errors
- Detection of undeclared variables